scientific reports

Check for updates

OPEN

Small defect detection in printed circuit boards based on the multiscale edge strengthening and an improved YOLOv10

Weixun Chen✉, Xuneng Ke & Siming Meng

The manufacturing processes of Printed Circuit Boards (PCBs) have become increasingly complex, and even minor defects can significantly impair product performance and yield. Accurate identification of PCB defects is therefore crucial but remains challenging. Given the low resolution, small target size, and diverse nature of PCB surface defects, this study proposes a novel YOLO MSES-SPDConv (MS-YOLO) network based on an improved YOLOv10 framework to achieve more accurate and efficient detection of minor PCB defects with a smaller model size. Firstly, to address the performance bottleneck of the C2F module in the head network of YOLOv10 when processing complex PCB features, we introduce the Multiscale Edge Strengthening (MSES) structure to replace the traditional C2F module. The MSES structure employs a multi-branch design for multidimensional learning of minor PCB features enhanced by edge information. During inference, it is simplified to a single branch to retain high-frequency information and reduce memory consumption. Secondly, in the neck of the network, we incorporate the Improved SPDConv module to minimize the loss of fine-grained information and enable multiscale feature extraction for PCB defect images. Lastly, we optimize the network structure using the Layer Adaptive Magnitude-based Pruning (LAMP) method and design a dual-distillation strategy to further refine the improved YOLOv10 algorithm. This strategy leverages two distinct network heads for knowledge transfer, enhancing the learning effectiveness of the student model and improving accuracy while reducing model size. Experimental results demonstrate that the proposed MS-YOLO outperforms several state-of-the-art models, achieving a mean Average Precision (mAP@50) of 98. 9%. This validates the significant improvements of MS-YOLO in detection accuracy, operational efficiency, and practical applicability.

Keywords PCB defect detection, MSES module, YOLOv10, SPDConv module, Dual-pruning strategy

Printed Circuit Board (PCB), as a core component of the electronics information industry, is increasingly utilized in fields such as computing, autonomous driving, aerospace, and other high-end technologies, making their quality assurance critically important[1]. The quality of PCB directly determines the performance and reliability of electronic devices. Consequently, enhancing PCB quality holds critical significance for the entire electronic information industry. With the growing market demand for precise and complex electronic circuits, the trend toward miniaturization and higher density of electronic components has made PCB more susceptible to various factors during production. These factors include mechanical friction, electrostatic interference, and chemical corrosion, which may lead to defects such as missing holes, mouse bites, open circuits, short circuits, burrs, and false copper[2].These issues not only affect the appearance of the PCB but may also degrade its performance and even lead to the failure of the circuit board. Defects may occur in different areas such as the substrate, copper surface, traces, and vias. Therefore, surface defect detection has become a basic requirement for quality control in PCB manufacturing.

Early detection of surface defects in printed circuit boards (PCBs) relied on manual inspection, functional testing[3], and in-line testing methods. Manual inspection is susceptible to visual fatigue, resulting in low efficiency and high error rates. Functional testing is costly and time-consuming, making it difficult to rapidly identify a variety of defects. In-line testing is limited to detecting electrical functional defects, with restricted detection range and precision[4]. With the advancement of technology, traditional image processing algorithms

The College of Artificial Intelligence, Guangzhou Railway Polytechnic, Guangzhou 510430, China. ✉email: chenweixun@gtxy.edu.cn

Content courtesy of Springer Nature, terms of use apply. Rights reserved

utilized visual features such as texture, edges, and contrast to identify defects[5,6]. However, their reliance on local features (e. g. , Scale-Invariant Feature Transform [SIFT][7]and Speeded-Up Robust Features [SURF][8]) in complex backgrounds limited their application effectiveness.

Machine learning algorithms enabled the automated detection of PCB defects by extracting features from visual images and employing classifiers[9]. Techniques such as Support Vector Machines (SVM), decision trees, and random forests have been widely applied, significantly improving detection accuracy and efficiency[10]. However, these methods often depend on effective feature extraction algorithms and still face challenges in handling complex foreground–background segmentation and detecting small targets[11].

In recent years, with the development and application of deep learning in various fields[12–17], researchers have introduced deep learning methods to PCB surface defect detection. Techniques such as candidate box generation, feature pyramid networks[18], and multiscale detection[19] have been employed to address issues related to small target detection[20] and complex texture interference to some extent. Deep learning-based object detection methods are primarily divided into one-stage and two-stage[21] algorithms. Representative two-stage algorithms include Faster R-CNN[22], DetectoRS[23], and Mask R-CNN[24]. These algorithms are characterized by high precision but have complex training processes, slow speeds, and large model sizes, making them unsuitable for lightweight and high-speed detection in industrial scenarios [25]. Representative one-stage algorithms include the YOLO series, SSD[26], EfficientDet[27], and RetinaNet[28]. These algorithms have simplified network structures, improved detection speed and accuracy[29], Among single-stage networks, the YOLO series[30–32] has been widely adopted in various downstream tasks, demonstrating advantages in detection accuracy, speed, and deployment compared to other models.

This study focuses on single-stage object detection algorithms, utilizing the YOLOv10 algorithm as the baseline model. In the global network architecture, we propose a Multiscale Edge Strengthening (MSES) structure, which employs a multi-branch design. During the training stage, the MSES structure enhances PCB features by incorporating edge information through multidimensional learning. In the inference stage, the structure is simplified to a single branch to retain high-frequency information and reduce memory consumption. In the neck network, we incorporate an improved SPDConv module[33], which processes features across multiple scales to effectively enhance the neck network's ability to extract features and resist noise in PCB defect images. Finally, to address the issue of increased model size after fusion and improvement, we apply the Learning-based Adaptive Model Pruning (LAMP)[34] score pruning method to globally remove the lowest-scoring connections until the desired sparsity is achieved. After pruning optimization, the model size is reduced by 15% compared to the original YOLOv10 model[35]. Based on this, we design a dual-distillation strategy for YOLOv10, which further improves model accuracy while reducing model space. This ultimately resolves the challenges of high difficulty in distinguishing PCB surface defects from the background, as well as the misclassification and missed detection caused by small defect sizes. The main contributions of this study are as follows:

1. We propsoe the MSES module that adopts a three-path cascaded architecture, combining the efficient feature extraction of the RepConv module and the dynamic edge enhancement method of the MCEE module. Through a single feature aggregation operation, it integrates the feature maps of each branch at the network's end, effectively capturing multiscale features and reducing redundancy. Experimental results demonstrate that the model with the MSES module achieves a 0. 9% improvement in the mAP@0. 5-0. 95 metric compared to models with the traditional C2F module.
2. We propose an improved SPDConv module. Compared with the original SPDConv, it divides the input feature map into two equal parts rather than four parts. Moreover, the original 1×1 is replaced by a 3×3 convolution for dimensionality reduction, which effectively preserves more feature information and enhancing the feature extraction capability for small targets. Experimental results show that the introduction of the Improved SPDConv module results in a 0.9% improvement in the mAP@0. 5-0. 95 metric.
3. To address the significant performance drop in pruned models, we design a dual-distillation strategy based on the pruned model. Unlike traditional one-to-one distillation methods, this strategy employs a dual-network head structure for knowledge transfer. The student model learns from a single teacher network through a one-to-one distillation branch and receives richer supervisory signals through a one-to-many distillation branch. This multi-faceted knowledge transfer approach helps the student network more effectively capture and imitate the key features of the teacher network. Experimental results indicate that the MS-YOLO model achieves a 2.1% improvement in the mAP@0. 5-0. 95 metric compared to the baseline model. The overall flowchart of the algorithm is shown in Fig. 1.

The remainder of this paper is organized as follows. The Related Work section presents research related to PCB surface defect detection. The Introduction to MS-YOLO section introduces the YOLOv10n framework as well as the network structure of the MS-YOLO model. The Methodology section provides a detailed introduction to the methods proposed in this paper. The Experiments section reports experimental data, procedures, results, and analysis of the results. Finally, in the Conclusion section, we further summarize and analyze the methods proposed in this paper and reflect on the work to be done next.

Related work

In this section, we first briefly review the related work of applying traditional visual algorithms, machine learning algorithms, and deep learning algorithms to PCB surface inspection, and analyze the advantages and disadvantages of each stage of PCB quality inspection. Subsequently, the detection performance of the model proposed in this paper is introduced.

In the research of traditional visual inspection algorithms for PCB detection, Zhu et al.[36] leveraged the superior performance of the Canny operator to automatically generate PCB edge maps, thereby enhancing

Content courtesy of Springer Nature, terms of use apply. Rights reserved

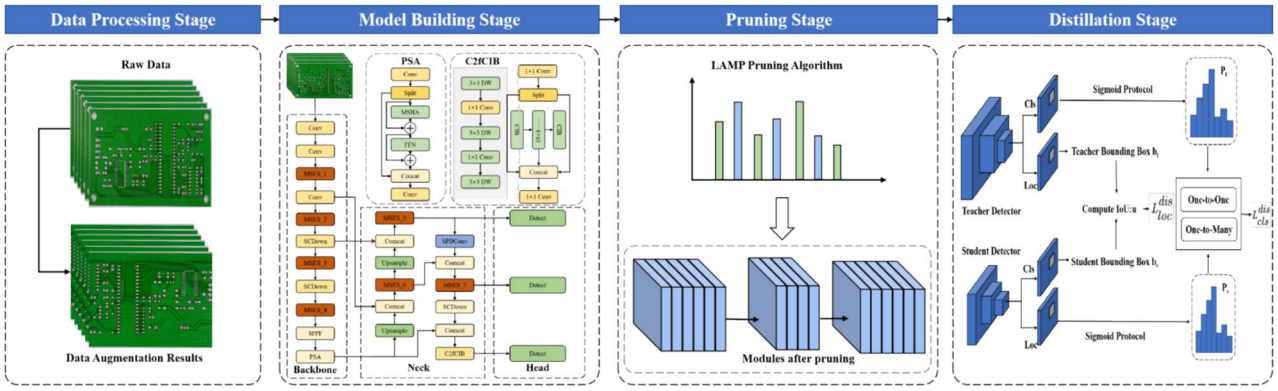


Fig. 1. The overall workflow of the algorithm in this study.

defect detection capabilities. Na Dong et al.[37] proposed an improved particle swarm optimization algorithm to address template matching problems, enabling accurate determination of fault locations and types. However, these traditional methods fall short in robustness and real-time performance, which are critical for industrial applications.

In the realm of machine learning-based PCB inspection algorithms, Hao Wu et al.[38] introduced a feature selection method and a two-stage classifier for solder joint inspection, utilizing tricolor hemispherical LED array illumination and a color digital camera to acquire solder joint images. Thanasis Vafeiadis et al.[10] proposed a framework for detecting PCB surface defects based on Support Vector Machine (SVM). While these methods have improved robustness, they still face challenges such as large data volumes, information redundancy, high-dimensional feature spaces, and susceptibility to environmental factors.

In the context of two-stage object detection networks for PCB inspection, Hu et al.[39] proposed an improved Faster RCNN algorithm to enhance the detection of small defects. Zhang et al.[40] introduced a cost-sensitive residual convolutional neural network (CSResNet) that effectively balances sample imbalance and misclassification costs. Although these algorithms achieve high precision, they are characterized by complex training procedures, slow detection speeds, and large model sizes, making them unsuitable for lightweight and high-speed detection requirements in industrial settings.

To address these limitations, researchers have developed single-stage object detection algorithms. For example, Wujin Jiang et al.[41] proposed the RAR-SSD algorithm, which enhances feature focusing through an attention mechanism to improve detection accuracy. Xia et al.[42] introduced an improved YOLO model incorporating a global context attention module to further enhance feature fusion. Kong et al.[43] developed the SMC-YOLO network based on YOLOv7-tiny, which significantly improves the detection accuracy of tiny defects but has a relatively large model size. Overall, these algorithms perform suboptimally in high-precision small target detection.

In summary, existing PCB quality inspection algorithms face three major challenges: (1) difficulty in accurately identifying tiny defects, leading to missed detections; (2) similarity between defect features and background colors, resulting in false detections; and (3) large model sizes, which are impractical for deployment on edge devices.

In light of the problems with existing PCB quality inspection algorithms, in order to improve the accuracy and detection speed of PCB surface defect recognition while meeting the requirements for lightweight deployment, this paper takes a one-stage object detection algorithm as the research direction and proposes an improved MS-YOLO algorithm with YOLOv10n as the baseline model. This algorithm deals with a large number of small targets and complex background noise interference on PCB surfaces. The algorithm achieves 98. 8% mAP@0. 5, 77. 9% mAP@0. 5:0. 95, 68. 0 FPS, and a model size of 4 Mb on the PCB dataset, verifying the effectiveness of the algorithm.

Methodology

YOLOv10 represents a significant milestone in the YOLO series, integrating the strengths of its predecessors and introducing numerous novel features and substantial enhancements. The series comprises six distinct model variants: YOLOv10n, YOLOv10s, YOLOv10m, YOLOv10b, YOLOv10l, and YOLOv10x. As the depth and width of these models increase, so too does the number of parameters and computational demands, catering to a wide range of applications from resource-constrained devices to high-performance computing platforms.In the context of PCB defect detection, where ease of deployment and real-time performance are critical, this study selects the YOLOv10n model as the baseline for PCB defect detection. The architecture of this model is primarily divided into three components: the backbone network responsible for feature extraction, the neck network for feature layer fusion, and the head network for generating classification and localization prediction outputs. As depicted in Fig. 2, we propose MS-YOLO by making global modifications to YOLOv10n. Specifically, we optimize all C2f. modules in the network to MSES modules and introduce our proposed the Improved SPDConv module into the neck network. Finally, we optimize the model using pruning and distillation techniques. These

Content courtesy of Springer Nature, terms of use apply. Rights reserved

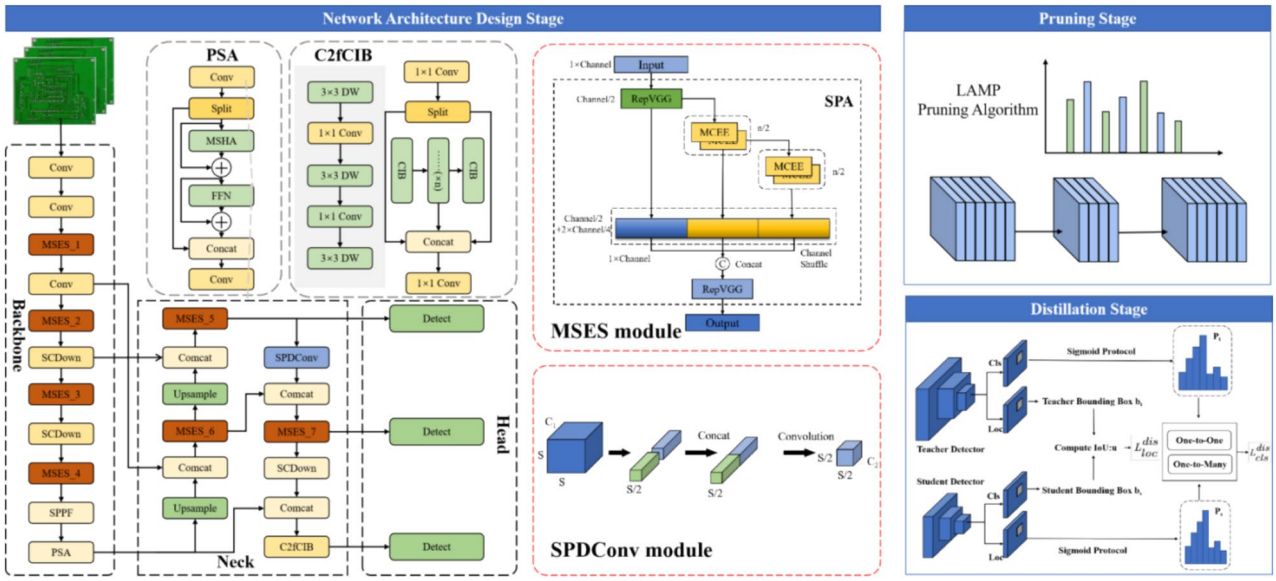


Fig. 2. The architectural design diagram of the MS-YOLO network.

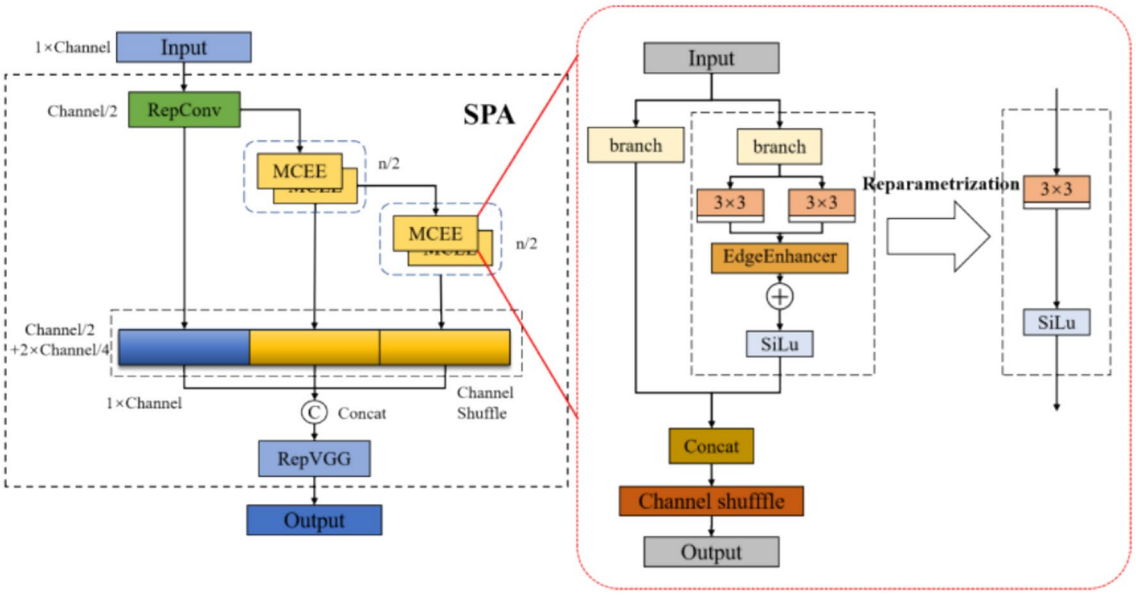


Fig. 3. Schematic diagram of the MSES architecture.

architectural modifications are designed to address the challenge of detecting small PCB defects against complex backgrounds, and will be detailed in subsequent sections.

The multiscale edge strengthening (MSES) module

To address the limitations of the C2F module in the YOLOv10 head network, particularly its overly simplistic design and performance bottlenecks when processing complex PCB features, this paper proposes a redesigned module called MSES (Multiscale Edge Strengthening). The MSES module enhances the model's ability to capture edge details at multiple scales and improves both the representation of complex features and detection accuracy through a combination of multiscale feature extraction and edge enhancement techniques. The MSES module consists of two key components: SPA (Single-Pass Aggregation) and MCEE (Multi-Channel Edge Enhancement). Figure 3 provides a schematic diagram of the MSES architecture.

As illustrated in Fig. 3, the PCB feature map is first input into the MSES module from the Input end, and then proceeds to the SPA module. The SPA module employs a three-path cascaded architecture to process the input features. This architecture is designed to capture features at different scales and levels through distinct pathways,

Content courtesy of Springer Nature, terms of use apply. Rights reserved

thereby enhancing the model's ability to perceive multiscale targets. The processing workflow of the three-path cascaded structure is as follows:

1) The first branch: This branch directly employs the RepConv module to perform initial feature extraction and channel adjustment on the input feature map. RepConv is an efficient module that integrates convolution and identity mapping, enabling robust feature extraction with minimal parameters. By dynamically adjusting the convolutional kernel and channel numbers, RepConv effectively extracts local features and optimizes the distribution of feature channels.
2) The second and third branches: These branches follow the same processing procedure, both utilizing stacked MCEE modules for feature extraction. The MCEE module is designed with residual connections, which enhance cross-channel information interaction between adjacent layers and enable the reuse of features across different levels.

Following processing by the three-path cascaded structure, the feature maps from each branch are integrated at the network's end through a single-pass feature aggregation operation. This operation enhances the model's ability to detect targets of varying sizes while reducing feature redundancy and storage requirements. During this process, channel shuffling is employed to recombine and rearrange all processed feature channels [44]. Channel shuffling disrupts fixed channel associations, promotes cross-feature interaction, and establishes more effective feature connections. Finally, the output feature map (Output) preserves the same size as the input feature map while demonstrating enhanced multiscale feature representation capabilities.

The MCEE module is inspired by the architectural features of the RepVGG network[45], employing distinct operations during training and inference to balance the richness of feature learning and the efficiency of inference. During training, the feature map is split through channel shuffling, and the feature map, enhanced by RepVGG processing and maintained in its original state, boosts the learning capability. During inference, structural reparameterization converts the dual-branch structure into a single convolutional kernel, reducing computational complexity and memory consumption to enable fast inference. As shown in Fig. 4, the feature dimension of the input tensor is first set to $C \times H \times W$. Following the channel splitting technique used in the ShuffleNet network, the input feature channels are divided into two separate channel dimensions, each sized $C \times H \times W$. The input feature channels are processed in two distinct stages: the training stage and the inference stage:

(1) Training stage: The MCEE module during training is constructed using two 3×3 convolutions and an EdgeEnhancer module. The EdgeEnhancer module scales the input 3×3 convolution to multiple scales and extracts local feature information at each scale. It then uses residual computation to subtract the average pooled features from the original features, effectively removing low-frequency background components and enhancing high-frequency edge features. Finally, cross-scale edge feature aggregation is achieved via channel concatenation.

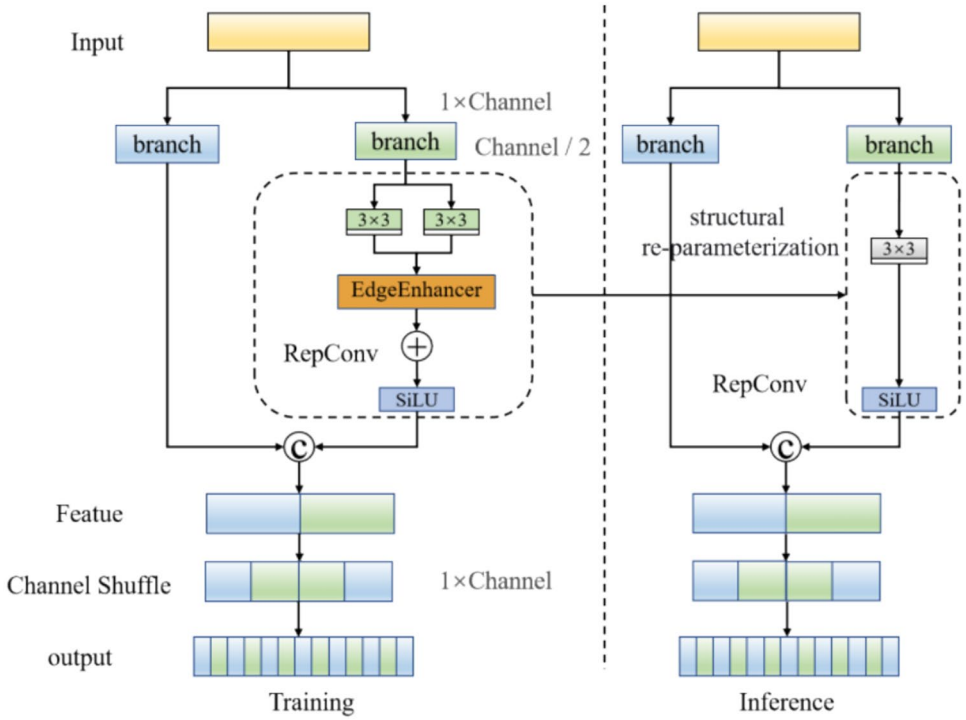


Fig. 4. Schematic diagram of the MCEE architecture.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

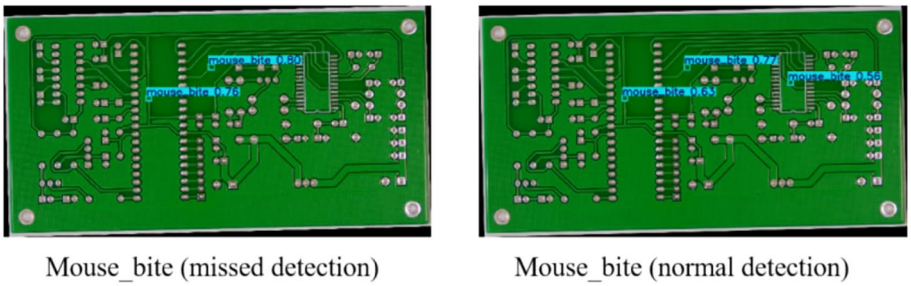


Fig. 5. Mouse_bite detection results comparison.

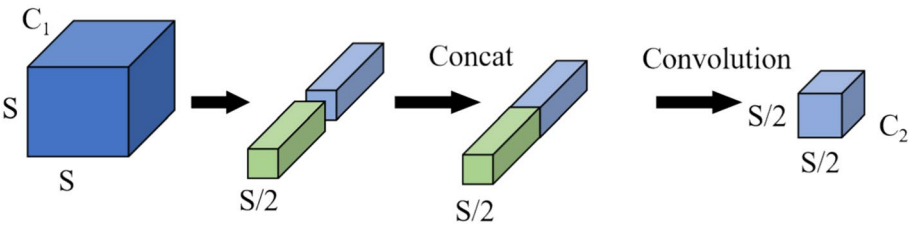


Fig. 6. Schematic diagram of the SPDConv architecture.

(2) Inference stage: By utilizing structural reparameterization techniques, the EdgeEnhancer module and the two 3 × 3 convolutions are converted into a single 3 × 3 RepConv. This approach reduces computational complexity and memory consumption during inference while preserving the feature representation capability acquired during training. The multi-branch topology learns rich feature representations during the training stage, while the simplified single-branch structure during inference saves memory consumption, enabling fast and efficient inference.

The improved spatial pyramid decomposition convolution (SPDConv)

Conventional PCB inspection algorithms frequently produce false detections when distinguishing between defect areas and non-defect zones, as exemplified in Fig. 5. This challenge primarily stems from the intrinsically minute scale of defect characteristics and their chromatic resemblance to background patterns, making them difficult to reliably identify with traditional inspection methods and leading to suboptimal accuracy[46]. To address this critical limitation, this research implements the Improved SPDConv module within the YOLOv10n framework. Comparative analyses reveal that the SPDConv-enhanced architecture demonstrates higher precision and superior robustness in small object detection relative to conventional convolution modules, particularly in recognizing subtle defects. The detailed architecture of this module is illustrated in Fig. 6.

The Improved SPDConv module is composed of a spatial partitioning module and a basic convolutional layer. The spatial partitioning module employs a specific downsampling strategy to divide the input feature map into two equally sized sub-feature maps, each with dimensions of $H/2 \times W/2 \times C$, where H, W, and C denote the height, width, and number of channels of the input feature map, respectively. These two sub-feature maps are subsequently concatenated along the channel dimension via a Concat module, resulting in a feature map of size $H/2 \times W/2 \times 2C$. A 3×3 convolution operation is then applied to reduce the dimensionality and establish connections between the feature maps, yielding a final feature map of size $H/2 \times W/2 \times C$. This process demonstrates that, compared to traditional convolutional operations, SPDConv can better preserve the information within the feature maps through its unique partitioning and combining approach, thereby enhancing the feature extraction capability for small-scale features.

When integrated into the YOLOv10n model, the Improved SPDConv module significantly improves the detection accuracy for low-resolution images and small targets. Specifically, the Backbone convolutional layers are retained, while the Neck layer is replaced with a downsampling convolutional module. Experimental results indicate that the model incorporating the Improved SPDConv module achieves a 0.9% improvement in the mAP@0.5–0.95 metric compared to the model using standard convolutional modules.

Dual distillation technique

Knowledge distillation is a model compression technique that leverages a complex and high-performing teacher model to guide the training of a simpler student model, thereby enhancing its performance while maintaining low parameter and computational requirements. In the training of YOLOv10, a dual-label assignment strategy is employed, which integrates both one-to-one and one-to-many label assignment methods. Building on this concept, this study proposes a dual-distillation strategy for YOLOv10, utilizing the BCKD (Bidirectional Channel Knowledge Distillation) algorithm to distill the improved YOLOv10 model. The unpruned model serves as the teacher model, providing knowledge in the form of output probability distributions to guide the training of the

Content courtesy of Springer Nature, terms of use apply. Rights reserved

student model, which has been pruned using the LAMP (Layer-Adaptive Magnitude-based Pruning) method. This guidance helps the student model improve its predictive accuracy. The BCKD distillation loss comprises two components: classification distillation loss and localization distillation loss.

During object detection, feature maps of size H × W × K are extracted, where H, W,and K represent height, width, and channel dimensions respectively. For input sample X, the teacher and student models generate probability values l^t and l^s which are subsequently processed through a sigmoid function to produce n × K dimensional outputs P^t and P^s.

When implementing the dual distillation strategy (one-to-one, one-to-many), this study performs two distillation steps for the prediction of each target:

One-to-one distillation strategy: A cross-entropy distillation loss function is used to measure the discrepancy between the student model's output and the teacher model's output (classification distillation), as illustrated in Eq. (1).

$$L_{cls}^{dis}(x) = \sum_{i=1}^{n}\sum_{j=1}^{K} L_{BCE}\left(p_{i,j}^{s}, p_{i,j}^{t}\right) \tag{1}$$

$L_{cls}^{dis}(x)$ denotes the classification distillation loss, with $p_{i,j}^{s}$ and $p_{i,j}^{t}$ representing the j-th class at the i-th position of p^s and p^t , respectively. To more effectively extract the most significant samples, the Focal loss approach is employed to weight the loss computation. For a sample *X*, its importance weight is $w \in R^{n\times K}$, and the formula for calculating w is presented in Eq. (2).

$$w = \left|p^t - p^s\right| \tag{2}$$

By weighting each element of w into the loss function, the final loss formula is derived, as presented in Eq. (3).

$$L_{cls}^{dis}(x) = \sum_{i=1}^{n}\sum_{j=1}^{K} w_{ij}w_{ij} \cdot L_{BCE}\left(p_{i,j}^{s}, p_{i,j}^{t}\right) \tag{3}$$

For the 1-to-N distillation strategy: In one-to-many distillation, a cross-entropy distillation loss function is initially employed to compute the distillation loss between the student model layer's output and the outputs of multiple teacher model layers. These losses are subsequently integrated. Given that the 1:N distillation strategy acts as a supervisory signal for the 1:1 distillation strategy, to prevent interference with the 1:1 strategy, a weight of 0. 5 is applied in the calculation of the final loss formula.

$$L_{cls}^{dis,1-to-many}(x) = \sum_{i=1}^{n}\sum_{k=1}^{M} 0.5 \cdot w_{ij} \cdot L_{BCE}\left(p_{i,j}^{s}, \frac{1}{M}\sum_{m=1}^{M} p_{i,j}^{t,m}\right) \tag{4}$$

Here, $\frac{1}{M}\sum_{m=1}^{M} p_{i,j}^{t,m}$ denotes the average of the probability values derived from the *M* layers of the teacher model, where $p_{i,j}^{t,m}$ represents the output of the m-th layer of the teacher model. In the context of one-to-many distillation, this average $\frac{1}{M}\sum_{m=1}^{M} p_{i,j}^{t,m}$ serves as a reference for computing the distillation loss of the student model's output $p_{i,j}^{s}$.

In localization distillation, the IoU between the bounding boxes output by the teacher and student models is utilized as the distillation target. For a sample *X* , the localization predictions at the i-th position by the teacher and student models are denoted as o_i^t and o_i^s respectively. Subsequently, the anchor positions and the obtained localization predictions are processed through the decoder to generate the predicted bounding boxes for sample *X*, as follows:

$$b_i^t = Decoder(A_i, o_i^t) \tag{5}$$

$$b_i^s = Decoder(A_i, o_i^s) \tag{6}$$

In Eqs. (5) and (6),A_i denotes the i-th anchor, b_i^t and b_i^s represent the predicted bounding boxes from the teacher and student models, respectively. The IoU between these two predicted bounding boxes is computed and denoted as u_i . The loss weighting strategy previously employed, which increases the weight for important samples, is then applied to the loss calculation in localization distillation, leading to Eq. (7).

$$L_{loc}^{dis}(x) = \sum_{i=1}^{n}\sum_{j=1}^{k} w_{i,j} \cdot (1 - u_i) \tag{7}$$

Content courtesy of Springer Nature, terms of use apply. Rights reserved

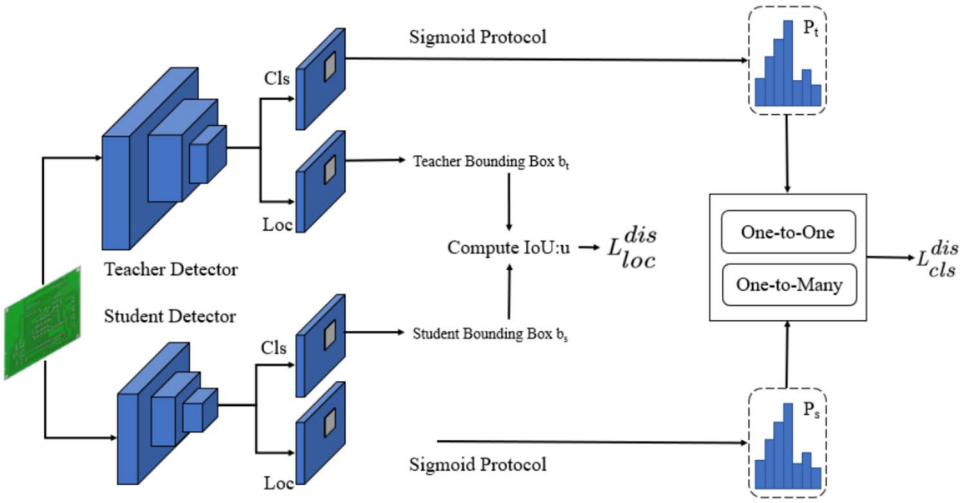


Fig. 7. Illustration of the dual distillation technique application.

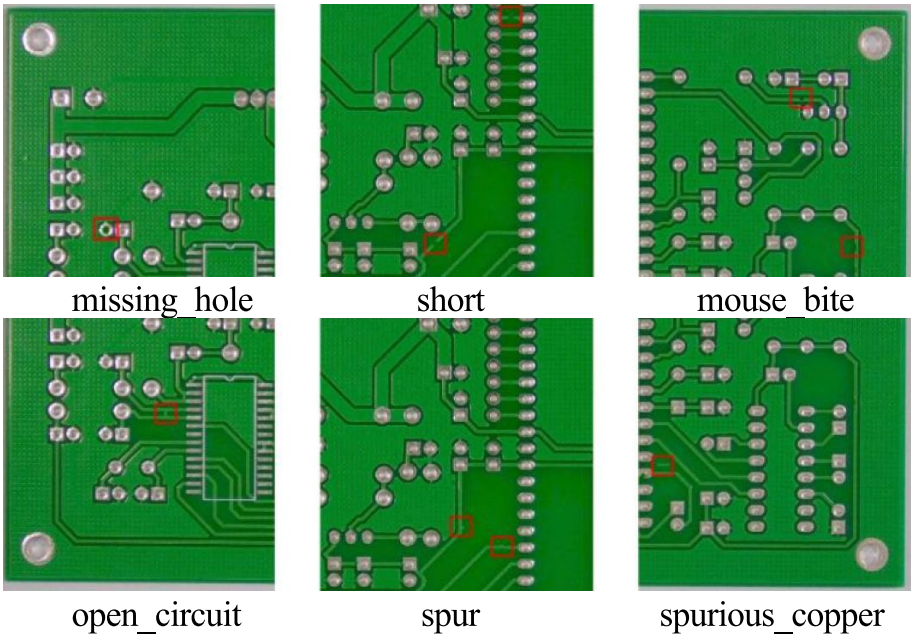


Fig. 8. Illustration of defect categories in the PKU-Market-PCB dataset.

After deriving the final classification distillation loss and the IoU-based localization distillation loss, these two losses are weighted and combined to improve the model's classification and localization performance. The total distillation loss formula is shown in Eq. (8).

$$L_{total}^{dis}(x) = a_1 \cdot L_{cls}^{dis}(x) + a_2 \cdot L_{loc}^{dis}(x) \tag{8}$$

The ultimate YOLOv10 algorithm training framework incorporating knowledge distillation is depicted in Fig. 7.

Experiments

Experimental dataset

The PKU-Market-PCB[47] is a synthetically generated PCB defect image dataset publicly released by Peking University. There are 1,386 images suitable for defect detection tasks, including 693 normally placed PCB images and 693 images that have undergone random rotation processing, with an average pixel size of 2777 × 2138. As shown in Fig. 8, there are six categories of PCB defects: missing hole, short, mouse bite, open circuit, spur, and spurious copper, and each defect image contains only one defect category.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

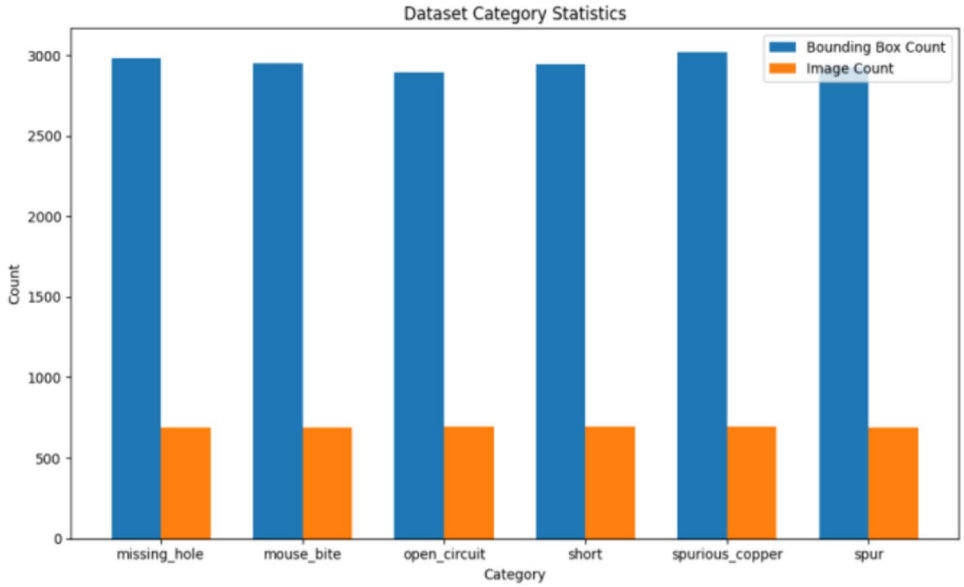


Fig. 9. Chart showing the number and image distribution of each category in the PKU-Market-PCB dataset.

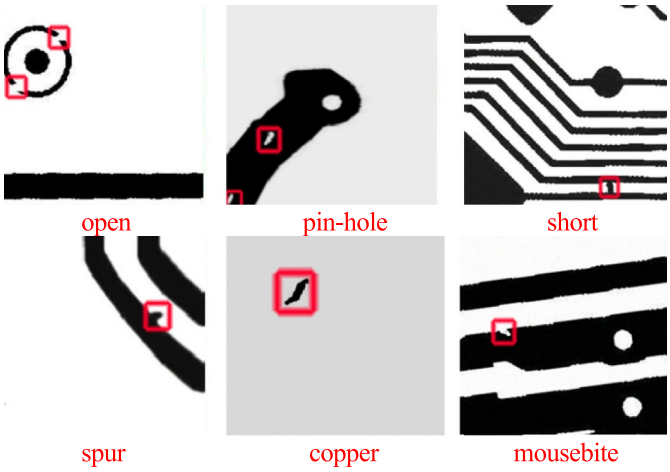


Fig. 10. Illustration of defect categories in the DeepPCB dataset.

This study conducts experiments using a PCB image dataset comprising 1,386 images. To simulate the complex scenarios that may arise during PCB quality inspection, the original dataset is augmented using image augmentation techniques such as Gaussian blur, random scaling, cropping, random brightness adjustment, salt-and-pepper noise, and random rotation. After augmentation, the dataset is expanded to 4,158 images. The distribution of various defects and instances in the dataset is depicted in the Fig. 9, The dataset is randomly split into training, validation, and testing sets in a 7:1:2 ratio based on the six defect categories, with 2,910 images in the training set, 416 in the validation set, and 832 in the testing set.

The DeepPCB dataset [48], released by the Institute of Image Processing and Pattern Recognition at Shanghai Jiao Tong University, is a publicly available PCB defect dataset. It comprises 1,500 defect images, encompassing six common PCB defect types: open, short, mousebite, spur, copper, and pin-hole, as detailed in Fig. 10. Given the scarcity of real image defects, this dataset incorporates artificially synthesized defects, with each image containing approximately 3 to 12 defects. This approach significantly expands the defect sample size. Additionally, binarization is applied to mitigate the impact of lighting and background variations.

Configuration environment and evaluation criteria

The model training platform used in this study was equipped with high-performance hardware, including an Intel Core i9-13900HX CPU, an NVIDIA GeForce RTX 4070 GPU, and 32 GB of GPU memory. The software environment was based on a 64-bit Windows 11 operating system, with Python 3.8 as the programming language.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Software	Detailed Information
OS	Windows11, https://www.microsoft.com/zh-cn/software-download/windows11
Programming language	Python 3.8, https://www.python.org/
Labelling software	LabelImg 1.8.6, https://pypi.org/project/labelImg/
Deep learning libraries	Torch 2.1.0 + cu118; Torchvision 0.16.0 + cu118, https://pytorch.org/; CUDA Version 11.7, https://developer.nvidia.com/cuda-toolkit-archive

Table 1. Software used in this study.

Training parameter	Value
Initial learning rate	0. 01
Optimizer	SGD
Optimizer momentum	0. 937
Weight decay coefficient	0. 0005
Training batch size	32
Number of epochs	300

Table 2. Training parameters.

The deep learning framework employed was PyTorch 2.1.0 + cu118, utilizing CUDA 11.7 for GPU acceleration. Detailed specifications are provided in Table 1, and the training parameters are summarized in Table 2.

This paper employs the Mean Average Precision (mAP), Precision, and Recall as evaluation metrics to comprehensively assess the performance of the detection model. The calculation formulas for these evaluation criteria are presented as follows:

$$\mathrm{Pr}\,ecision = \frac{TP}{TP + FP} \tag{9}$$

$$\mathrm{Re}call = \frac{TP}{TP + FN} \tag{10}$$

In Eqs. (9) and (10), True Positives (TP) refer to the number of positive samples correctly identified, False Positives (FP) denote the number of negative samples incorrectly predicted as positive, and False Negatives (FN) represent the number of positive samples incorrectly predicted as negative. Precision and recall are complementary metrics for evaluating the performance of a classification model. Precision reflects the proportion of actual positive samples among those predicted as positive, while recall measures the proportion of actual positive samples that are successfully identified by the model. Relying solely on either metric is insufficient for a comprehensive evaluation of model performance.

$$AP = \frac{\sum_{n=1}^{N} p(n) * gt(n)}{N_{gt}} \tag{11}$$

$$mAP = \frac{\sum_{m=1}^{Q} p(n) * AP(m)}{Q} \tag{12}$$

In Eqs. (11) and (12), AP (Average Precision) is calculated by determining the precision and recall values of the positive samples predicted by the model at different confidence thresholds, plotting the PR (Precision-Recall) curve, and then averaging the area under the PR curve. mAP (mean Average Precision) represents the average of the individual class AP values.

Ablation experiments

To evaluate the impact of the improved modules on the model, this study employed an ablation study strategy by incrementally adding modules. The evaluation criteria included computational cost, parameter count, model size, mAP@0. 5, and mAP@0. 5–0. 95. In the experiments, Model1 was the original YOLOv10n model. Model2 replaced the C2F module in Model1 with the MSES module, which used a multi-branch architecture to extract and enhance edge information in PCB features. Model3 introduced the Improved SPDConv module to Model2 to reduce the loss of fine-grained features and enhance the ability to capture multiscale features. Model4 was derived from Model3 by applying a pruning ratio of 1.4 and using a dual-distillation strategy with one-to-one and one-to-many distillation paths to improve model accuracy. The optimized model was thus obtained. The results of the ablation study are shown in Table 3.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Model	MSES	SPDConv	Prune	Distill	GFLOPs/G	Params/M	Weight/MB	mAP@0. 5/%	mAP@0. 5–0. 95/%
1					6. 5	2. 2	5. 6	98. 7	77.4
2	√				18. 2	4. 7	11. 1	98. 9	78.3
3	√	√			18.4	4.8	11	98.9	79.2
4	√	√	√		13.2	2.8	6.6	98.7	78.4
5	√	√	√	√	13.2	2.8	6.6	98.7	79.5

Table 3. Ablation study on the PKU-Market-PCB dataset.

Model	Precision/%	Recall/%	mAP@0. 5/%	mAP@0. 5–0. 95/%	Params/M	FPS
YOLOv5n	97. 8	96. 1	97. 7	57. 5	3. 8	181
YOLOv8n[49]	90. 4	87. 8	97. 9	70. 3	6. 1	155
YOLOv9t[50]	98. 7	96. 1	98. 2	74. 2	4. 5	162
YOLOv10n[35]	90. 8	95. 9	98. 7	76. 9	5. 6	176
YOLOv11n[51]	98. 1	95. 3	97. 3	69. 2	5. 3	165
GCC-YOLO[42]	99. 0	97. 3	98. 2	69. 8	14. 8	90
MS-YOLO(Ours)	98. 8	97. 5	98. 9	79.5	4. 7	164

Table 4. Training results of different algorithms on the PKU-Market-PCB dataset.

As shown in Table 3, the incorporation of the MSES module into Model 1 resulted in an increase in the mAP@0.5–0.95 metric from 77.4% to 78.3%. This improvement indicates that edge enhancement significantly benefits the detection of minor defects on PCBs. However, this enhancement also led to an increase in computational load by 11.7 GFLOPS and an increase in the number of parameters by 2.5 M.For Model 3, the application of the Improved SPDConv module further improved the mAP@0.5–0.95 metric by 0.9%. Nevertheless, this improvement was accompanied by increases in computational load, number of parameters, and model size. At this stage, Model 3's mAP@0.5–0.95 metric was 1.8% higher than that of the baseline Model 1, but all other parameters were also higher than those of the baseline Model 1.Model 4, which underwent LAMP score pruning and sparse training to recover precision, experienced reductions of 28%, 42%, and 40% in computational load, number of parameters, and model size, respectively. However, the mAP@0.5–0.95 metric decreased to 78.4%. To address the loss of key features in the model due to pruning, we employed a dual distillation strategy, allowing the student model (Model 4) to relearn from the teacher model (Model 3, which had not undergone sparse training).The results demonstrated that Model 5, after distillation learning, achieved a significant improvement in the mAP@0.5–0.95 metric compared to the previous improved models, indicating more accurate detection of small target PCB defects. Although Model 5 exhibited increases in the number of parameters, computational load, and weight size compared to the YOLOv10n baseline model, these increases were within an acceptable range and were necessary to achieve higher detection accuracy.

Comparative experiment

To evaluate the detection performance of MS-YOLO for small targets under complex background interference, this paper selected MS-YOLO and other advanced object detection models such as YOLOv5n, YOLOv8n[49], YOLOv9t[50], YOLOv10n[35], YOLOv11n[51], and GCC-YOLO[42]for performance comparison. To ensure the fairness of the comparison, all models were trained on the same dataset. The comparative experimental data is shown in Table 4.

As shown in Table 4, a comparative analysis of various network models was conducted, focusing on mean average precision (mAP@0.5–0.95), model complexity (Params), and inference speed (FPS). Our proposed MS-YOLO model achieved a detection accuracy of 79.5% in terms of mAP@0.5–0.95, representing a 22.0% improvement over the earlier YOLOv5n model. When compared with YOLOv8n (70.3%), YOLOv9t (74.2%), YOLOv10n (76.9%), and YOLOv11n (69.2%), our model demonstrated respective increases of 9.2%, 5.3%, 2.6%, and 10.3% in mAP@0.5–0.95. Additionally, our model outperformed the similarly optimized GCC-YOLO model (69.8%) by 9.7% in precision.In terms of model complexity, MS-YOLO has only 4.7 M parameters, which is a 22.9% reduction compared to YOLOv8n (6.1 M) and a 68.2% reduction compared to GCC-YOLO (14.8 M). When compared to models with similar parameter scales, MS-YOLO achieved a 5.3% improvement in mAP@0.5–0.95 while increasing the parameter count by only 4.4% relative to YOLOv9t (4.5 M). Compared to YOLOv5n (3.8 M), our model increased the parameter count by 23.7% but achieved a 38.3% higher detection accuracy, thereby achieving a favorable balance in lightweight design that enhances detection accuracy without significantly increasing model complexity.Although our model is not the smallest in terms of size, it achieves the optimal balance between accuracy, speed, and size. While maintaining high detection accuracy, our model reaches an inference speed of 164 FPS, surpassing GCC-YOLO (90 FPS) and YOLOv11n (165 FPS). Even when compared to the lightweight YOLOv5n model (181 FPS), our model still maintains a certain speed advantage, meeting the real-time detection requirements for edge devices.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

As shown in Fig. 11, the detection results comparison between the MS-YOLO algorithm and YOLO series algorithms for six PCB defect types is presented. Each row uses the same image, with corresponding defect types including missing holes, mouse bites, open circuits, short circuits, burrs, and false copper. There are significant differences in the detection performance of MS—YOLO and other advanced object detection algorithms on the PCB dataset. GCC-YOLO performed the worst on this dataset, with six false positives and one false negative. Other algorithms also failed to completely avoid false positives and false negatives. Among them, YOLOv5, YOLOv8, and YOLOv11 each had two targets missed, while YOLOv9 and YOLOv10 failed to detect three targets. In terms of false positives, YOLOv8, YOLOv9, and YOLOv10 each had two, while YOLOv5 and YOLOv11 each had one. The comprehensive analysis results show that MS—YOLO exhibits higher detection accuracy for defect categories such as mouse bites and burrs compared to other algorithms, indicating that MS -YOLO has a clear advantage in identifying these small defects.

Comparative experiments of MS-YOLO on the DeepPCB dataset

To further validate the generalizability of MS-YOLO, identical experiments were conducted on another publicly available dataset, DeepPCB.The experimental results comparing MS-YOLO with other YOLO models are presented in Table 5.

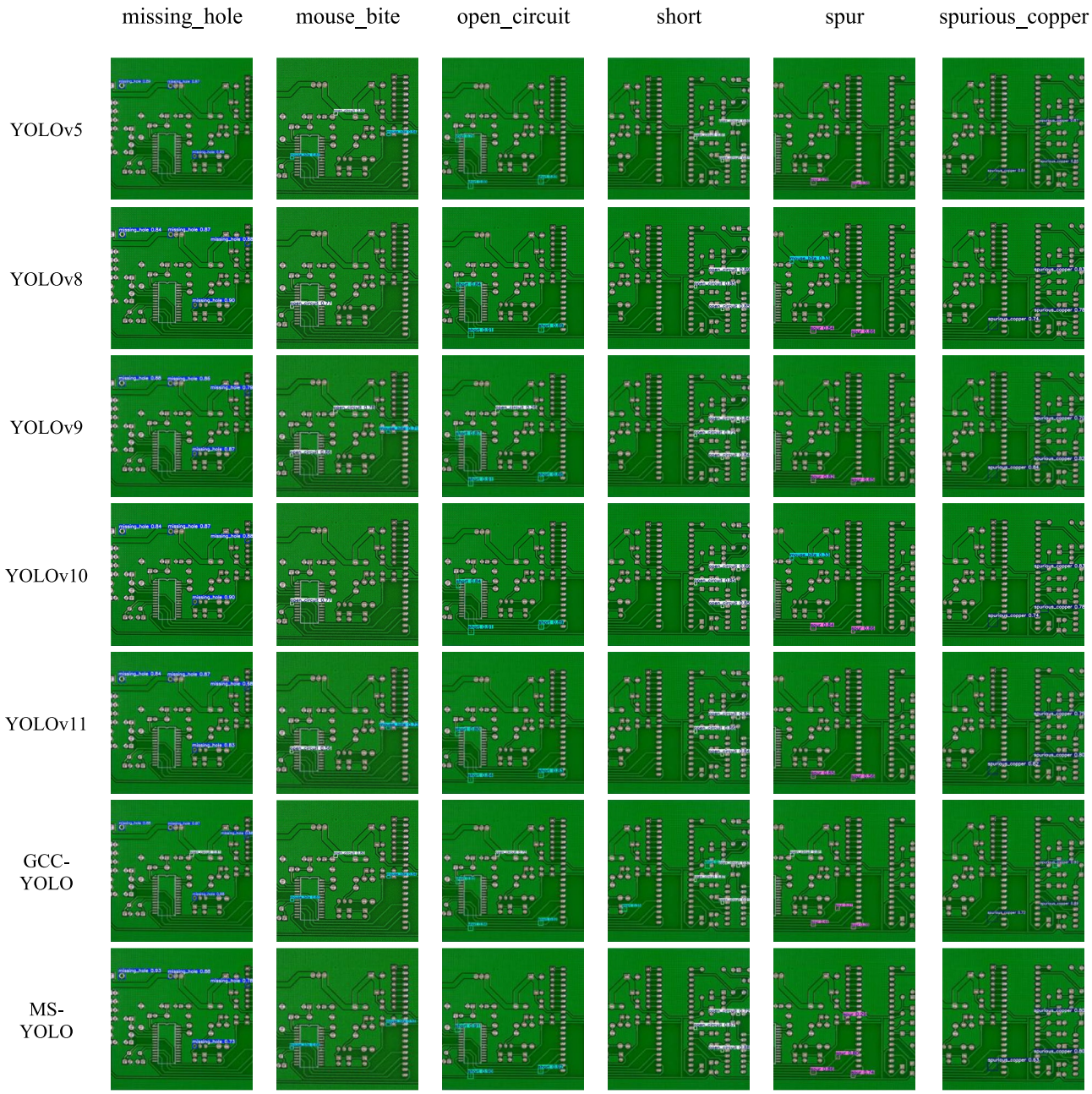


Fig. 11. Comparison of detection results of different algorithms.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Model	Precision/%	Recall/%	mAP@0. 5/%	mAP@0. 5–0. 95/%	Params/M	FPS
YOLOv5n	98.2	94.7	97.5	67.5	3.8	184
YOLOv8n	96.7	90.1	94.7	64.1	6.1	157
YOLOv9t	98.4	95.8	95.7	67.2	4.5	164
YOLOv10n	97.1	90.2	95.4	67.3	5.6	173
YOLOv11n	95.3	90.4	94.7	66	5. 3	163
GCC-YOLO	96.3	92.8	96.7	67.5	14.8	94
MS-YOLO (Ours)	96.4	91.7	96.4	68.5	4.7	164

Table 5. Comparative experiments of the MS-YOLO model on the DeepPCB dataset.

As shown in Table 5, MS-YOLO outperforms various versions of YOLO models, including YOLOv5n, YOLOv8n, YOLOv9t, YOLOv10n, YOLOv11n, and GCC-YOLO, in terms of detection accuracy. Specifically, MS-YOLO achieves a 1.0, 4.4, 2.8, 3.2, 2.5, and 1.0 percentage point increase in mAP@0.5–0.95 compared to these models, respectively. In terms of mAP@0.5, MS-YOLO reaches 97.7%, surpassing all other YOLO series models.Regarding model complexity, MS-YOLO has 4.7 M parameters, which is significantly lower than YOLOv8n (6.1 M) and GCC-YOLO (14.8 M), but higher than YOLOv5n (3.8 M) and YOLOv9t (4.5 M). In terms of inference speed, MS-YOLO achieves an FPS of 164, second only to YOLOv5n and YOLOv10n. This indicates that MS-YOLO maintains high accuracy while also offering fast inference speed, making it suitable for applications with high real-time requirements.

More Experimental details regarding hyperparameters setting, ablation study on the DeepPCB dataset, etc., can be found in the Supplementary Material S1.

Conclusion

This study presents the MS-YOLO model, an enhanced YOLOv10n network structure tailored to address the challenges of small target detection on the PCB dataset. Initially, the traditional C2F module is replaced with the MSES structure, which employs a multi-branch design to enrich feature learning and simplifies to a single branch during inference to minimize memory usage. Subsequently, the Improved SPDConv module is integrated into the head network to reduce the loss of fine-grained information and bolster the extraction of multiscale features from PCB defect images. Finally, a score-based pruning method is applied to eliminate redundant information and compress the model size. Additionally, a dual-label distillation strategy for YOLOv10 is designed to further enhance model accuracy. Experiments demonstrate that MS-YOLO outperforms other mainstream object detection models in mitigating false positives and false negatives. It achieves higher detection accuracy and speed while satisfying the requirements for lightweight deployment, thus meeting the industrial demands for PCB surface defect detection.

Data availability

The datasets generated during and analyzed during the current study are available from the corresponding author on reasonable request.

Received: 21 April 2025; Accepted: 15 September 2025
Published online: 17 October 2025

References

1. Xiao, G., Hou, S. & Zhou, H. PCB defect detection algorithm based on CDI-YOLO. *Sci. Rep.* **14**(1), 7351 (2024).
2. Yuan, M. et al. YOLO-HMC: an improved method for PCB surface defect detection. *IEEE Trans. Instrum. Meas.* **73**, 2001611 (2024).
3. Lv, S. et al. A dataset for deep learning based detection of printed circuit board surface defect. *Scientific Data* **11**(1), 811 (2024).
4. Zhou, Y. et al. Review of vision-based defect detection research and its perspectives for printed circuit board. *J. Manuf. Syst.* **70**, 557–578 (2023).
5. Ma, H. Y. et al. Automated void detection in high resolution x-ray printed circuit boards (PCBs) images with deep segmentation neural network. *Eng. Appl. Artif. Intell.* **133**, 108425 (2024).
6. Tang, J. et al. A lightweight surface defect detection framework combined with dual-domain attention mechanism. *Expert Syst. Appl.* **238**, 121726 (2024).
7. Lowe, D. G. Distinctive image features from scale-invariant keypoints. *Int. J. Comput. Vision* **60**(2), 91–110 (2004).
8. Bay, H. et al. Speeded-up robust features (SURF). *Comput. Vis. Image Underst.* **110**(3), 346–359 (2008).
9. Pham, T. T. et al. Defect detection in printed circuit boards using semi-supervised learning. *Sensors* **23**(6), 3246 (2023).
10. Thanasis Vafeiadis, N. D. D. I. & Tzovaras, D. A framework for inspection of dies attachment on PCB utilizing machine learning techniques. *J. Manag. Anal.* **5**(2), 81–94 (2018).
11. Kiobya, T., Zhou, J. & Maiseli, B. A multi-scale semantically enriched feature pyramid network with enhanced focal loss for small-object detection. *Knowl.-Based Syst.* **310**, 113003 (2025).
12. Dlamini, S., Kuo, C. J. & Chao, S. Developing a surface mount technology defect detection system for mounted devices on printed circuit boards using a MobileNetV2 with feature pyramid network. *Eng. Appl. Artif. Intell.* **121**, 105875 (2023).
13. Liu, X. et al. Gaussian-IoU loss: Better learning for bounding box regression on PCB component detection. *Expert Syst. Appl.* **190**, 116178 (2022).
14. Liu, Q. et al. A real-time anchor-free defect detector with global and local feature enhancement for surface defect detection. *Expert Syst. Appl.* **246**, 123199 (2024).

Content courtesy of Springer Nature, terms of use apply. Rights reserved

15. Yu, X. et al. PCB defect detection model based on intrinsic feature decomposition and multilevel fusion against image uncertainty. *IEEE Sens. J.* **24**(12), 19497–19505 (2024).
16. Li, X. et al. Energy-propagation graph neural networks for enhanced out-of-distribution fault analysis in intelligent construction machinery systems. *IEEE Internet Things J.* **12**(1), 531–543 (2025).
17. Li, X. et al. Adaptive expert ensembles for fault diagnosis: A graph causal framework addressing distributional shifts. *Mech. Syst. Signal Process.* **234**, 112762 (2025).
18. Yu, J. et al. Defect detection of printed circuit board based on adaptive key-points localization network. *Comput. Ind. Eng.* **193**, 110258 (2024).
19. Yu, X. et al. Progressive refined redistribution pyramid network for defect detection in complex scenarios. *Knowl.-Based Syst.* **260**, 110176 (2023).
20. Zhang, L. et al. LDD-Net: Lightweight printed circuit board defect detection network fusing multi-scale features. *Eng. Appl. Artif. Intell.* **129**, 107628 (2024).
21. Wang, C., Wei, X. & Jiang, X. An automated defect detection method for optimizing industrial quality inspection. *Eng. Appl. Artif. Intell.* **127**, 107387 (2024).
22. Ren, S. et al. Faster R-CNN: towards real-time object detection with region proposal networks. *IEEE Trans. Pattern Anal. Mach. Intell.* **39**(6), 1137–1149 (2017).
23. Qiao, S., L. Chen and A. Yuille, DetectoRS: Detecting objects with recursive feature pyramid and switchable atrous convolution. Preprint at: arXiv:2006.02334 (2020).
24. He K, Gkioxari G, Dollár P, Girshick R., Mask r-cnn. In: *Proceedings of the IEEE International Conference on Computer Vision*, 2017 (2961–2969).
25. Wang, Z. & Shen, X. SD2Net: surface-mounted device detection network with convolution-free attention mechanism for printed circuit board integrity assurance. *Appl. Intell.* **53**(20), 23582–23595 (2023).
26. Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., Berg, A.C. 2016: SSD: Single shot multibox detector. In: *Proceedings of the 14th European Conference on Computer Vision (ECCV 2016)*, (11–14) (21–37).
27. Tan, M., R. Pang and Q.V. Le, 2020 EfficientDet: *Scalable and Efficient Object Detection*., 10778–10787
28. Lin, T.; Goyal, P.; Girshick, R.; He, K.; Dollár, P. Focal Loss for Dense Object Detection. *In Proceedings of the 2017 IEEE International Conference on Computer Vision (ICCV)*, (22–29) (2999–3007).
29. Xia, Z. et al. FC-DETR: High-precision end-to-end surface defect detector based on foreground supervision and cascade refined hybrid matching. *Expert Syst. Appl.* **266**, 126142 (2025).
30. Yu, Z. et al. A lightweight and efficient model for surface tiny defect detection. *Appl. Intell.* **53**(6), 6344–6353 (2023).
31. Zhang, Y. et al. DsP-YOLO: An anchor-free network with DsPAN for small object detection of multiscale defects. *Expert Syst. Appl.* **241**, 122669 (2024).
32. Chen, L. et al. YOLO-DHGC: small object detection using two-stream structure with dense connections. *Sensors* **24**(21), 6902 (2024).
33. Sunkara, R.; Luo, T. No more strided convolutions or pooling: A new CNN building block for low-resolution images and small objects. In: *Proceedings of the Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD* 2022, Grenoble, France, 19–23 2022 443–459.
34. Lee, J., et al., Layer-adaptive sparsity for the Magnitude-based Pruning. Preprint at: arXiv:2010.07611. (2020)
35. Wang, A.; Chen, H.; Liu, L.; Chen, K.; Lin, Z.; Han, J.; Ding, G., YOLOv10: Real-Time End-to-End Object Detection. Preprint at: arXiv:2405.14458 (2024).
36. Zhu, W., Gu, H. & Su, W. A fast PCB hole detection method based on geometric features. *Meas. Sci. Technol.* **31**(9), 095402 (2020).
37. Dong, N. et al. Chaotic species based particle swarm optimization algorithms and its application in PCB components detection. *Expert Syst. Appl.* **39**(16), 12501–12511 (2012).
38. Wu, H. et al. Classification of solder joint using feature selection based on bayes and support vector machine. *IEEE Trans. Components, Packaging Manufact. Technol.* **3**(3), 516–522 (2013).
39. Hu, B. & Wang, J. Detection of PCB surface defects with improved faster-RCNN and feature pyramid network. *IEEE Access* **8**, 108335–108345 (2020).
40. Zhang, H., Jiang, L. & Li, C. CS-ResNet: Cost-sensitive residual convolutional neural network for PCB cosmetic defect detection. *Expert Syst. Appl.* **185**, 115673 (2021).
41. Jiang, W. et al. PCB defects target detection combining multi-scale and attention mechanism. *Eng. Appl. Artif. Intell.* **123**, 106359 (2023).
42. Xia, K. et al. Global contextual attention augmented YOLO with ConvMixer prediction heads for PCB surface defect detection. *Sci. Rep.* **13**(1), 9805 (2023).
43. Kong, W. et al. SMC-YOLO: Surface Defect Detection of PCB Based on Multi-Scale Features and Dual Loss Functions. *IEEE Access* **12**, 137667–137682 (2024).
44. Zhang, X., et al., ShuffleNet: An extremely efficient convolutional neural network for mobile devices, Preprint at : arXiv:1707.01083 (2017).
45. Ding, X.; Zhang, X.; Ma, N.; Han, J.; Ding, G.; Sun, J. Repvgg: Making vgg-style convnets great again. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Nashville, TN, USA*, (20–25) (13733–13742) (2021)
46. Zhang, D. et al. A novel deep convolutional neural network algorithm for surface defect detection. *J. Comput. Design Eng.* **9**(5), 1616–1632 (2022).
47. Huang W, Wei P. A PCB dataset for defects detection and classification, Preprint at :arXiv:1901.08204 (2019).
48. Tang, S., et al., Online PCB defect detector on A new PCB defect dataset , Preprint at: arXiv:1902.06197 2019.
49. Varghese, R. and S. M., YOLOv8: A novel object detection algorithm with enhanced performance and robustness. In: *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)* (2024) (1–6).
50. Wang, C., I. Yeh and H. Liao, YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. *In Proceedings of the European Conference on Computer Vision (ECCV)*, (2024) (1–21).
51. Khanam, R. and M. Hussain, YOLOv11: An overview of the key architectural enhancements. Preprint at: arXiv:2410.17725 2024.

Author contributions

Weixun Chen designed the idea and led the writing of the article, Xuneng Ke conducted the experiment and collected the data. Siming Meng conducted theanalysis and supervised the whole process. All authors read and approved the final manuscript.

Funding

This work was supported by the Project of Young Innovative Talents of General Colleges and Universities of Guangdong Province under Grant 2024KQNCX212 and the Tertiary Education Scientific Research Project of Guangzhou Municipal Education Bureau under Grant 2024312419

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Supplementary Information The online version contains supplementary material available at https://doi.org/10.1038/s41598-025-20387-x.

Correspondence and requests for materials should be addressed to W.C.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).
Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.
These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.
We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.
While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.
These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.
To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.
Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.
If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com